

SYSTEM DESIGN

Chapter 51: Design a URL Shortener

Personal Notes

The URL shortener (bit.ly, tinyurl, goo.gl) is the canonical "hello world" of system design interviews. It's simple enough to fully design in 45 minutes, but rich enough to touch every major concept from Chapter 50 — capacity estimation, API design, data modeling, hashing, caching, database choice, sharding, and scaling. Nail this one and you'll be prepared for the entire genre of "design X" interviews.

This chapter walks through the design from start to finish using the 7-step framework. Along the way we'll compare three approaches to generating short codes (hash vs counter vs random), pick a database (SQL vs NoSQL — it matters here), design for a 100:1 read:write ratio, and handle the deep-dive questions interviewers love to ask: "What if two writes generate the same code?" "How would you scale to 100× the traffic?" "What happens when a shard dies?"

1. The Problem

Build a service that converts long URLs into short URLs (e.g., `https://example.com/very/long/path/to/article` → `https://short.ly/abc123`), and redirects visitors from the short URL back to the original long URL.

```
User submits:  https://example.com/some/very/long/path/here?
ref=xyz&id=42
      |
      ▼
Service returns: https://short.ly/abc123

Later – visitor clicks: https://short.ly/abc123
      |
      ▼
Server responds with:  HTTP 302 Redirect → original URL
```

2. Step 1 — Requirements Gathering

Always start here. In an interview, spend 5-8 minutes clarifying before touching the whiteboard.

2.1 Functional Requirements

- Create a short URL from a long URL
- Redirect a short URL to the original long URL
- (Optional) Custom aliases (e.g., `short.ly/my-brand`)
- (Optional) URL expiration (auto-delete after N days)
- (Optional) Analytics — click counts, geo distribution

2.2 Non-Functional Requirements

- Scale — assume ~100M new URLs/month, ~10B total, and ~100× reads vs writes
- Latency — redirects must be FAST (<100 ms p99) since they're on the click path
- Availability — 99.9%+ uptime; being down blocks every existing link
- Durability — links can't disappear; users share them permanently
- Consistency — eventual is OK for analytics, but the short→long mapping should be strong

2.3 Constraints to Confirm

- Short URL length — is 6-8 characters acceptable? Determines encoding space.
- Read/write ratio — usually 100:1 for URL shorteners (people share once, others click many times)
- Expiration — do links live forever, or expire after some time?
- Authentication — anonymous, or user accounts?

Interview tip: The read/write ratio is one of the most important numbers to establish. 100:1 read-heavy tells you the entire architecture should optimize for READS — caching, replicas, CDN. If it were 1:1 or write-heavy, the design would look totally different.

3. Step 2 — Capacity Estimation

Now the math. Order-of-magnitude only, but every subsequent decision is grounded in these numbers.

Write QPS

```
100M new URLs / month
÷ 30 days / month
÷ 86,400 seconds / day
= ~40 writes/sec (average)
× 3 (peak factor)
= ~120 writes/sec peak    ← very small
```

Read QPS

```
Writes × 100 (read:write ratio) = ~4,000 reads/sec average
                                = ~12,000 reads/sec peak
```

Read latency budget: p99 < 100 ms → redirect path must be fast

Storage

```
10 billion URLs total (5 years × 100M/month × 60 months × 2 buffer)
Per record: ~500 bytes (short_code + long_url + timestamps +
metadata)
Total: 10B × 500 B = 5 TB
```

Fits on one large machine, but we'll shard for HA + write throughput.

Bandwidth

Reads: $12,000 \text{ QPS} \times \sim 500 \text{ B} = 6 \text{ MB/sec}$ of DB reads
Writes: $120 \text{ QPS} \times 500 \text{ B} = 60 \text{ KB/sec}$

Neither is a bottleneck. Caching will cut read bandwidth further.

What the numbers tell us: This is a READ-HEAVY, LOW-QPS, HIGH-DURABILITY system. Caching will handle most reads. Storage is modest. Nothing screams "we need Kafka and Kubernetes" — a well-designed monolith with a cache and DB replication would work. This framing beats over-engineering.

4. Step 3 — API Design

Two endpoints. Keep it small — extra endpoints are noise.

```
POST /api/shorten
Content-Type: application/json

Request:
{
  "long_url": "https://example.com/very/long/path",
  "custom_alias": "my-brand",           // optional
  "expires_at": "2026-12-31"           // optional
}

Response:
{
  "short_url": "https://short.ly/abc123",
  "short_code": "abc123",
  "expires_at": "2026-12-31"
}

-----

GET /:short_code
Example: GET /abc123

Response:
HTTP/1.1 302 Found
Location: https://example.com/very/long/path
```

Why 302, Not 301?

- 301 (Permanent Redirect) — browsers CACHE the redirect. Faster on repeat visits, but breaks analytics (server never sees the click).
- 302 (Found, Temporary) — browsers do NOT cache. Every click hits your server → accurate analytics + ability to change destination.

- Choose 302 if analytics matter. 301 if maximum performance and analytics don't.

5. Step 4 — Data Model

The schema is small. This is what most tables look like:

Table: urls

Column	Type	Notes
short_code	VARCHAR(10)	PRIMARY KEY
long_url	TEXT	The original URL (~2000 chars max)
user_id	BIGINT	Optional — for authenticated users
created_at	TIMESTAMP	For analytics, cleanup
expires_at	TIMESTAMP	NULL = never expires
click_count	BIGINT	Denormalized counter (or track separately)

Index on short_code (unique) — the read path.
Optional index on user_id — for "my URLs" lookup.
Optional index on expires_at — for TTL cleanup.

Should short_code be primary key? Yes — every read is by short_code, and it's unique. No need for a separate auto-increment id. This also simplifies sharding: shard by short_code hash and every read goes to exactly one shard.

6. Step 5 — The Short-Code Algorithm

This is where most of the interesting discussion happens. THREE main approaches — each with distinct trade-offs.

6.1 Approach A — Hash the Long URL (MD5/SHA)

```
String shortCode = base62Encode(md5(longUrl)).substring(0, 7);
```

Pros: deterministic (same URL always yields same code), no coordination between servers needed.

Cons: COLLISIONS — different URLs can hash to the same short code. Deterministic also means users can't get a DIFFERENT short URL if they want it. Also — same URL by different users returns the same code, which may leak information.

The collision problem: MD5 has 128 bits. After truncating to 7 base62 characters (~42 bits), the birthday paradox kicks in at ~2 million URLs. At 10 billion, collisions are guaranteed. You'd have to handle them — retry with a suffix, use more characters, or fall back to another method.

6.2 Approach B — Counter + Base62 Encoding

Maintain a GLOBAL COUNTER. When a new URL comes in, get the next counter value, encode it in base62 (0-9, a-z, A-Z = 62 characters), and use that as the short code.

```
public String shortCode(long id) {
    StringBuilder sb = new StringBuilder();
    String chars =
"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    while (id > 0) {
        sb.append(chars.charAt((int)(id % 62)));
        id /= 62;
    }
    return sb.reverse().toString();
}

// Counter = 12345678 → shortCode = "u90G"    (4 chars)
// Counter = 62^7 - 1 → shortCode = "zzzzzzz" (7 chars, ~3.5 trillion
// IDs)
```

- Pros: NO collisions (counter is unique), short + sequential, easy to understand.
- Cons: counter becomes a SINGLE POINT OF CONTENTION at high write QPS. Also: sequential codes are guessable (enumerate short URLs).

The Counter Bottleneck — How to Solve It

- Central counter with atomic increment (Redis INCR) — simple, works up to ~50K QPS, single point of failure
- Ticket server (Flickr's approach) — a dedicated service hands out ranges of IDs to workers
- ZooKeeper coordination — for a globally consistent counter across nodes
- Sharded counters — each shard has its own range (worker 1: 1-1M, worker 2: 1M-2M, ...)

6.3 Approach C — Random Generation + Uniqueness Check

```
String code;
do {
    code = generateRandom7Chars();
} while (db.exists(code));    // retry if collision
db.insert(code, longUrl);
```

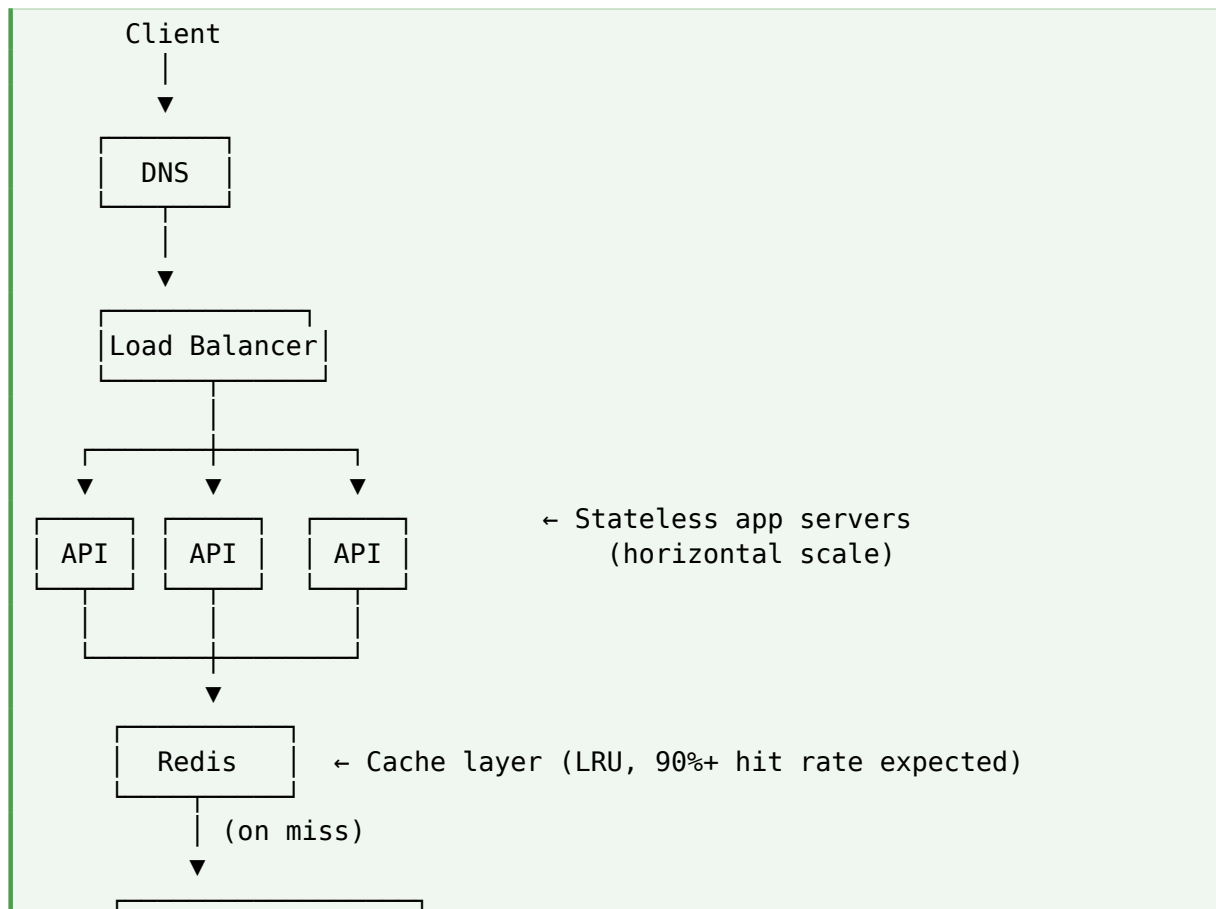
- Pros: no counter contention, codes are unguessable, works across many servers with no coordination.
- Cons: extra DB lookup per generation (usually cheap), potential retry loop when address space fills up.

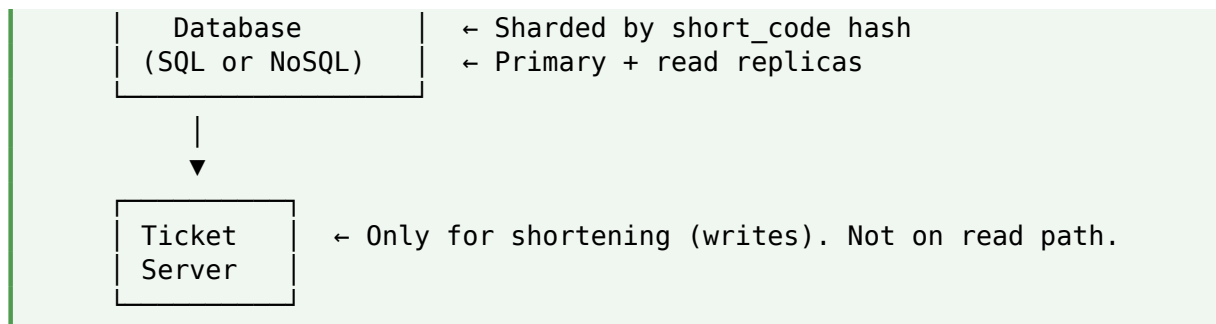
Which to Pick?

Approach	Verdict
Hash	Simple + deterministic, but collisions inevitable at scale. Rarely picked in interviews without justification.
Counter + base62	Best if you can solve the counter bottleneck. Common Google/interview answer. Combine with a ticket server for scale.
Random + check	Great for small-to-medium scale, unguessable codes. Retry cost negligible while address space is > 20% empty.

Interview-worthy answer: "I'd use a counter + base62 encoding, but instead of a single counter I'd use a ticket server that hands out ranges of 1000 IDs to each app instance. This way each instance can generate 1000 URLs without coordination, and the ticket server handles maybe 1 request per 1000 URL creations — no bottleneck." This walks the interviewer through both the naive answer AND the scale-up.

7. Step 6 — High-Level Architecture





Request Flow — Redirect (the hot path)

1. Client requests GET /abc123
2. LB routes to an available app server
3. App checks Redis for short_code="abc123"
4. Cache HIT (90%+ of time) → return 302 Redirect immediately (~5 ms total)
5. Cache MISS → query DB shard → populate Redis → return 302 (~30-50 ms)

Request Flow — Shorten (rare, once per URL)

6. Client posts long URL to /api/shorten
7. App gets next ID from ticket server (may batch)
8. App encodes ID to base62 → short_code
9. App writes (short_code, long_url) to DB and cache
10. App returns short URL to client

8. Step 7 — Deep Dive: Caching

With 100:1 read:write and 10B URLs, caching is the biggest lever. Get the caching layer right and the database barely sweats.

What to Cache

- The short_code → long_url mapping (the ONLY thing the redirect path needs)
- TTL: 24 hours is a good default — most link clicks concentrate right after sharing
- Eviction: LRU — rarely-used old links get pushed out naturally

Cache Sizing

Assume 20% of URLs get 80% of clicks (Pareto).
Hot set: ~20% of 10B = 2 billion URLs.
Per entry: ~500 bytes (code + URL + overhead).
Total memory: $2B \times 500 B = 1 TB$ — too much for RAM.

Realistic: cache the top 5-10% of active URLs.
 $500M \text{ URLs} \times 500 B = 250 GB$ → fits in ~5 large Redis instances.

Expected hit rate: 90-95% given click concentration.

Cache Strategy

CACHE-ASIDE (from Ch 50) is the natural fit. On a read, check cache first; on miss, hit DB and populate cache. This keeps the cache fresh with actively-used entries and lets the LRU eviction handle staleness.

Thundering herd on cache miss: If a super-popular URL expires from cache at the same moment thousands of clicks arrive, they'll all hit the DB simultaneously. Fix: use SINGLE-FLIGHT (only one request per key fetches from DB, others wait) or a distributed lock. Skipping this can cause DB overload during traffic spikes.

9. Deep Dive: Database Choice

Aspect	SQL (PostgreSQL, MySQL)	NoSQL (Cassandra, DynamoDB)
Data model	Simple key-value pattern	Simple key-value pattern
Consistency	Strong	Eventual, tunable
Scale writes	Sharding is manual + hard	Native — DynamoDB, Cassandra scale to millions of QPS
Read latency	Fast with index, replicas help	Very fast (Cassandra/DynamoDB single-digit ms)
Operational cost	Well-understood tooling	Cassandra requires ops expertise
Interview default	OK for small scale (up to ~10K QPS)	Better for large scale — this problem qualifies

The senior answer: "Given the simple key-value access pattern, high read QPS, and 10B rows, I'd pick a NoSQL wide-column store like Cassandra or a managed key-value store like DynamoDB. If we were smaller (say 100M URLs and 1K QPS), a PostgreSQL primary with a couple of read replicas would work fine and be simpler to operate." This shows you can justify the choice at both scales.

10. Deep Dive: Sharding

With 5 TB of data and heavy read traffic, sharding is inevitable. The shard key choice determines everything.

Shard by short_code hash

- EVERY read query knows the short_code — perfect for a single-shard lookup
- Writes distribute evenly (short_codes are essentially random)
- No hot shards unless one link goes viral (and even then, cache absorbs it)

Alternative — Shard by user_id

- Fits "list my URLs" queries better
- But redirect path (the hot one!) needs to route by short_code, which loses locality
- Verdict: don't do this unless "list my URLs" is a top-tier requirement

Consistent Hashing

For the shard mapping, use CONSISTENT HASHING (Ch 50). When you add a new shard, only $\sim 1/N$ of keys need to move — as opposed to nearly all keys with naive $\text{hash}\%N$.

11. Deep Dive: Handling Collisions

Even with a counter approach, edge cases exist. With hash or random approaches, collisions are common.

Collision Detection

```
String generateShortCode(String longUrl) {
    for (int attempt = 0; attempt < 5; attempt++) {
        String code = generateCandidate(longUrl, attempt);
        if (db.putIfAbsent(code, longUrl)) {
            return code;    // atomic insert succeeded
        }
        // else: collision, try again
    }
    throw new CollisionException("Could not generate unique code");
}
```

The race condition to watch: Simple check-then-insert has a race — two threads see "no entry for code X," both try to insert. Use PUT-IF-ABSENT (INSERT ... ON CONFLICT in PostgreSQL, conditional put in DynamoDB). Never do check-then-insert without atomicity — silent data loss.

12. Additional Features

12.1 Custom Aliases

Users want `short.ly/my-brand` instead of `short.ly/x8Kp92`. Check availability, reserve the code, store with a "custom" flag. Reject if taken. Handle length limits (say, 4-20 characters) and disallow reserved words ("admin", "api", etc.).

12.2 URL Expiration

Store `expires_at` with each URL. On read, check the timestamp before returning. For cleanup, run a background job nightly that deletes expired entries — or use a TTL feature (Cassandra, Redis, DynamoDB all support this natively).

12.3 Analytics

For click counts, don't increment the URLs table directly (too much write contention on hot URLs). Instead: PUBLISH click events to a queue like Kafka; async workers aggregate into a separate analytics store (Cassandra, ClickHouse, BigQuery). The redirect path stays fast, and analytics can be rich (geo, referrer, user-agent, timestamps).

13. Security Concerns

Open Redirect Abuse

Attackers can shorten malicious URLs and trick users. Mitigations: scan destinations against threat-intel APIs (Google Safe Browsing, VirusTotal); block known-bad domains; sample-check high-volume shortenings; add captcha to prevent bot-driven mass creation.

Rate Limiting

Cap URL creation per IP / per user (e.g., 100/hour). Prevents abuse and mass link generation. Redirect path also benefits from rate limiting to prevent bot-driven click fraud.

Privacy

If codes are sequential (counter-based), attackers can enumerate ALL URLs by incrementing. Fix: use non-sequential codes (random or hashed) or add signature bits. Some services deliberately obfuscate to prevent scraping.

14. Failure Modes

Failure	Mitigation
Cache down	Every read hits DB → 100× DB load. Have circuit breaker; degrade gracefully.
DB shard down	That shard's URLs fail. Use replicas (auto-failover). Consistent hashing minimizes fault radius.
Ticket server down	Can't create new URLs, but reads still work. Have a backup ticket server + local ID pool per app instance.
Traffic spike (viral URL)	Cache absorbs it — single hot key can serve millions of QPS from Redis + CDN edges.
Region outage	Multi-region deployment: each region has its own stack, replicated writes.

Bring these up proactively: Interviewers love when candidates raise failure scenarios without being asked. Even if you don't have a perfect answer, saying "if the ticket server dies, we're blocked on

writes — I'd add a backup or pre-allocate ID ranges to each app instance" demonstrates operational maturity.

15. Bottleneck Progression — Scaling to 100×

Interviewers may push: "What if traffic grew 100×?" Walk through what breaks first and how to fix it.

11. At 10× — read cache handles it, small DB scale-up. No architectural change.
12. At 25× — write QPS hits ~3K/sec, ticket server contention shows up. Add batched ID allocation per app instance.
13. At 50× — DB shards near capacity, add more shards. Consistent hashing minimizes remapping.
14. At 100× — read QPS = 1.2M/sec. Move to CDN edge for redirect + multi-region deployment. Redis clusters per region.
15. Beyond — the design starts looking like Google URL Shortener's actual architecture: edge Points-of-Presence, regional caches, replicated global state.

16. Trade-offs Summary

Decision	Trade-off
302 vs 301 redirect	302 = accurate analytics; 301 = better performance, no click tracking
Counter vs random codes	Counter = short + no collisions; random = unguessable, no coordination
SQL vs NoSQL	SQL simpler ops for small scale; NoSQL for 10B+ rows with heavy reads
Cache TTL long vs short	Long = higher hit rate; short = fresher (matters if you support URL updates)
Sync analytics vs async	Sync = immediate counts but slow redirect; async (queue) = fast redirect, delayed counts
Custom aliases yes vs no	Yes = user-friendly; no = simpler system, no reserved-word conflicts

17. Common Mistakes

Mistake 1: Not clarifying the read:write ratio

Missing the 100:1 ratio leads to designs optimized for writes (weird). Always establish this ratio — it's the #1 architectural driver for URL shorteners.

Mistake 2: Suggesting SQL primary with no replicas

At 12K read QPS, a single primary is fine — but any real design needs read replicas (or a cache) for HA. Not mentioning either is a red flag.

Mistake 3: Not addressing cache invalidation

If URLs never change, invalidation is easy (TTL-based eviction). But if the interviewer says "users can edit their long URLs," you now need cache invalidation on update. Address it explicitly.

Mistake 4: Assuming a counter is trivial to scale

A single atomic counter caps at maybe 50K QPS (Redis INCR). Above that you need range allocation (ticket server) or ZooKeeper. Show awareness of the limit.

Mistake 5: Forgetting collision handling

Any hash-based or random approach has collisions. Handling them with PUT-IF-ABSENT is mandatory — a simple check-then-insert has a race condition.

Mistake 6: Ignoring the 302 vs 301 discussion

Redirect type has real consequences (analytics, browser caching, SEO). If the interviewer doesn't bring it up, YOU should — signals product/UX thinking.

Mistake 7: Not proposing a phased rollout

"I'd start with a simple monolith + PostgreSQL + Redis, then shard when we hit 5 TB, then add multi-region at 10× QPS." This staged thinking beats "here's the final architecture for 100M QPS" — real systems evolve.

18. Best Practices

- Start with requirements + capacity — 10 minutes minimum
- Establish the read:write ratio (100:1 for URL shortener)
- Use base62 encoding for short codes — $62^7 \approx 3.5$ trillion combinations
- Cache aggressively — 90%+ hit rate is realistic for this workload
- Use 302 redirects unless perf is critical AND analytics don't matter
- Shard by short_code hash — aligns with the hot read path
- Use PUT-IF-ABSENT for inserts to prevent race conditions
- Discuss failure modes proactively — cache down, DB shard down, ticket server down
- End with a staged growth plan — how the design evolves at 10×, 100×

19. Quick Reference

Concept	Meaning
Short URL	6-8 chars usually — base62 encoded
Base62	0-9, a-z, A-Z = 62 chars per position

Concept	Meaning
7 base62 chars	$62^7 \approx 3.5$ trillion possible codes
Read:write ratio	~100:1 for URL shorteners
Cache hit target	90-95% with LRU + 24h TTL
302 vs 301	302 = analytics-friendly; 301 = browser-cached
Counter approach	Simple + short; needs ticket server at scale
Random approach	Unguessable; needs uniqueness check
Hash approach	Deterministic but has collisions
Ticket server	Hands out ID ranges — solves counter bottleneck
Shard key	short_code hash — perfect for read path
PUT-IF-ABSENT	Atomic insert; prevents race on collision
Thundering herd	Many requests miss cache same key — use single-flight

20. Related Interview Problems

Once you've internalized URL shortener, these problems reuse the same patterns:

16. Design a Pastebin — same data model, different content sizes; discuss text-storage patterns.
17. Design a Video Metadata Service — same key-value pattern at larger scale; discuss CDN integration.
18. Design a Filesystem Metadata Service — like URL shortener, but with directory hierarchy.
19. Design a Coupon Code Generator — reuses counter/random code generation.
20. Design a Unique ID Generator (Snowflake) — deep dive into the ticket/coordination piece.
21. Design a Bookmarking Service — URL shortener + user accounts + folders.
22. Design a QR Code Service — extends URL shortener with image generation and caching.

21. Final Takeaway

URL shortener is deceptively deep. The design is simple — one key-value table, a cache, some app servers. But every design decision has a REAL TRADE-OFF: 302 vs 301, hash vs counter vs random, SQL vs NoSQL, shard key choice, TTL length, custom aliases yes/no. Answering "which do you pick and why" for each is what separates a strong candidate from an average one.

Practice this design until you can walk through it in 30 minutes cleanly. Then extend it: pretend the interviewer says "we want click analytics — how does that change your design?" or "we need custom

domains — what do you add?" or "traffic just went 100× in one day — where do you look first?" These "add-a-feature" prompts are extremely common and test your ability to modify a design under pressure.

Key Takeaway: URL shortener = READ-HEAVY key-value system (100:1 read:write). Framework: requirements → capacity → API → data model → high-level → deep dives. Short code = base62 encoding, 6-8 chars. Three generation approaches: HASH (collisions), COUNTER (needs ticket server at scale), RANDOM (needs uniqueness check). CACHE aggressively — 90%+ hit rate expected. NoSQL wins at 10B+ rows; SQL fine for smaller scale. SHARD by short_code hash — aligns with hot read path. Use PUT-IF-ABSENT to avoid races. Prefer 302 for analytics.

22. What's Next?

Now that you've done the canonical warm-up, next up:

- Design a News Feed / Twitter — timeline generation (fan-out on write vs read), celebrity problem
- Design a Chat System — WebSocket architecture, presence, message delivery guarantees
- Design a Rate Limiter — token bucket implementation at scale, distributed coordination
- Design a Distributed Cache — internals of Redis/Memcached, consistent hashing, replication
- Design a Notification System — push vs pull, delivery guarantees, template management
- Deep dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep dive: Kafka Architecture — partitions, consumer groups, exactly-once delivery
- Deep dive: Distributed Consensus — Raft algorithm, leader election, log replication